A Technical Introduction to Vanishing gradient problem

Section 1

x t = F (x t – 1 , u t , θ) = W rec σ (x t – 1) + W in u t + b {\displaystyle x_{t}=F(x_{t-1},u_{t},\theta)=W_{\text{rec}}\sigma (x_{t-1})+W_{\text{in}}u_{t}+b} where θ = (W rec , W in) {\displaystyle \theta =(W_{\text{rec}},W_{\text{in}})} is the network parameter, σ {\displaystyle \sigma } is the sigmoid activation function, applied to each vector coordinate separately, and b {\displaystyle b} is the bias vector. Then, ∇ x F (x t – 1 , u t , θ) = W rec diag ■ (σ ′ (x t – 1)) {\displaystyle \nabla _{x}F(x_{t-1},u_{t},\theta)=W_{\text{rec}}\operatorname {diag} (\sigma '(x_{t-1}))} , and so

Section 2

Recurrent network model A generic recurrent network has hidden states h 1 , h 2 , … {\displaystyle h_{1},h_{2},\dots } , inputs u 1 , u 2 , … {\displaystyle u_{1},u_{2},\dots } , and outputs x 1 , x 2 , … {\displaystyle x_{1},x_{2},\dots } . Let it be parameterized by θ {\displaystyle \theta } , so that the system evolves as

Section 3

Consider first the autonomous case, with u = 0 {\displaystyle u=0} . Set w = 5.0 {\displaystyle w=5.0} , and vary b {\displaystyle b} in [– 3 , – 2] {\displaystyle [-3,-2]} . As b {\displaystyle b} decreases, the system has 1 stable point, then has 2 stable points and 1 unstable point, and finally has 1 stable point again. Explicitly, the stable points are (x , b) = (x , ln ■ (x 1 – x) – 5 x) {\displaystyle (x,b)=\left(x,\ln \left({\frac {x}{1-x}}\right)-5x\right)} . Now consider Δ x (T) Δ x (0) {\displaystyle {\frac {\Delta x(T)}{\Delta x(0)}}} and Δ x (T) Δ b {\displaystyle {\frac {\Delta x(T)}{\Delta b}}} , where T {\displaystyle T} is large enough that the system has settled into one of the stable points. If (x (0) , b) {\displaystyle (x(0),b)} puts the system very close to an unstable point, then a tiny variation in x (0) {\displaystyle x(0)} or b {\displaystyle b} would make x (T) {\displaystyle x(T)} move from one stable point to the other. This makes Δ x (T) Δ x (0) {\displaystyle {\frac {\Delta x(T)}{\Delta x(0)}}} and Δ x (T) Δ b {\displaystyle {\frac {\Delta x(T)}{\Delta b}}} both very large, a case of the exploding gradient. If (x (0) , b) {\displaystyle (x(0),b)} puts the system far from an unstable point, then a small variation in x (0) {\displaystyle x(0)} would have no effect on x (T) {\displaystyle x(T)} , making Δ x (T) Δ x (0) = 0 {\displaystyle {\frac {\Delta x(T)}{\Delta x(0)}}=0} , a case of the vanishing gradient. Note that in this case, Δ x (T) Δ b ≈ ∂ x (T) ∂ b = (1 x (T) (1 – x (T)) – 5) – 1 {\displaystyle {\frac {\Delta x(T)}{\Delta b}}\approx {\frac {\partial x(T)}{\partial b}}=\left({\frac {1}{x(T)(1-x(T))}}-5\right)^{-1}} neither decays to zero nor blows up to infinity. Indeed, it's the only well-behaved gradient, which explains why early researches focused on learning or designing recurrent networks systems that could perform long-ranged computations (such as outputting the first input it sees at the very end of an episode) by shaping its stable attractors. For the general case, the intuition still holds (Figures 3, 4, and 5).

Section 4

Often, the output x t {\displaystyle x_{t}} is a function of h t {\displaystyle h_{t}} , as some x t = G (h t) {\displaystyle x_{t}=G(h_{t})} . The vanishing gradient problem already presents itself clearly when x t = h t {\displaystyle x_{t}=h_{t}} , so we simplify our notation to the special case with:

Section 5

x t + 1 = (1 – ε) x t + ε σ (w x t + b) + ε w ′ u t {\displaystyle x_{t+1}=(1-\varepsilon)x_{t}+\varepsilon \sigma (wx_{t}+b)+\varepsilon w'u_{t}}

Section 6

Weight initialization Weight initialization is another approach that has been proposed to reduce the vanishing gradient problem in deep networks. Kumar suggested that the distribution of initial weights should vary according to activation function used and proposed to initialize the weights in networks with the logistic activation function using a Gaussian distribution with a zero mean and a standard deviation of 3.6 / N {\displaystyle 3.6/{\sqrt {N}}} , where N {\displaystyle N} is the number of neurons in a layer. Recently, Yilmaz and Poli performed a theoretical analysis on how gradients are affected by the mean of the initial weights in deep neural networks using the logistic activation function and found that gradients do not vanish if the mean of the initial weights is set according to the formula: max (– 1 , – 8 / N) {\displaystyle \max(-1,-8/N)} . This simple strategy allows networks with 10 or 15 hidden layers to be trained very efficiently and effectively using the standard backpropagation.

Section 7

In machine learning, the vanishing gradient problem is the problem of greatly diverging gradient magnitudes between earlier and later layers encountered when training neural networks with backpropagation. In such methods, neural network weights are updated proportional to their partial derivative of the loss function. As the number of forward propagation steps in a network increases, for instance due to greater network depth, the gradients of earlier weights are calculated with increasingly many multiplications. These multiplications shrink the gradient magnitude. Consequently, the gradients of earlier weights will be exponentially smaller than the gradients of later weights. This difference in gradient magnitude might introduce

instability in the training process, slow it, or halt it entirely. For instance, consider the hyperbolic tangent activation function. The gradients of this function are in range [0,1]. The product of repeated multiplication with such gradients decreases exponentially. The inverse problem, when weight gradients at earlier layers get exponentially larger, is called the exploding gradient problem. Backpropagation allowed researchers to train supervised deep artificial neural networks from scratch, initially with little success. Hochreiter's diplom thesis of 1991 formally identified the reason for this failure in the "vanishing gradient problem", which not only affects many-layered feedforward networks, but also recurrent networks. The latter are trained by unfolding them into very deep feedforward networks, where a new layer is created for each time-step of an input sequence processed by the network (the combination of unfolding and backpropagation is termed backpropagation through time).